

# 0.98.1. Une lettre qui tire — plusieurs tirs, depuis ax et ay

Le 0.98 (la place du A dans ax et ay), avec les trois tirs du 0.97.1 : chaque tir part de  $ax + 1$ , sur la ligne ay.

## 1. Ce qu'il faut comprendre

**C'est le 0.98, avec plusieurs tirs.** À l'écran, c'est le 0.97.1 : le bouton A tenu lance une rafale, trois tirs au plus. Dans le programme, la place du A est dans `ax` et `ay` (le 0.98).

**Ce qui change par rapport au 0.97.1 :** partout où il y avait `2`, il y a `ax` ; partout où il y avait `8`, `ay`. Et le départ d'un tir est calculé : `tx[i] = ax + 1`.

**Pourquoi c'est utile :** au 0.99.1, le A va bouger. Chaque tir partira alors de là où est le A **au moment du tir**.

**Essaie :** `uint8_t ax = 0;` et `uint8_t ay = 3;` : le A et sa rafale descendent sur la ligne 3.

### Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)

## 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **674 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

|                   |       |                                                                        |
|-------------------|-------|------------------------------------------------------------------------|
| démarrage         | 60 o  | ← la console, à l'allumage — la mise en route de la console            |
| chaque_minuteur() | 124 o | ← main() — une fonction fournie par le langage                         |
| main()            | 269 o | ← démarrage — ta fonction                                              |
| VBlank            | 8 o   | ← la console, à chaque image — compte les images, 60 fois par seconde  |
| AttendreImage     | 28 o  | ← image() — attend l'image suivante                                    |
| AttendreVBlank    | 16 o  | ← démarrage — attend que l'écran soit libre                            |
| AttendreAcces     | 12 o  | ← EffacerCases, EcrireTexte — attend de pouvoir écrire une case        |
| AdresseCase       | 10 o  | ← poser(), texte(), effacer() — trouve l'adresse d'une case de l'écran |
| EffacerCases      | 12 o  | ← effacer() — efface des cases                                         |
| EcrireTexte       | 10 o  | ← texte() — écrit un texte, lettre par lettre                          |
| CopierTuiles      | 40 o  | ← démarrage — copie les lettres et les dessins en mémoire vidéo        |
| EffacerCarte      | 14 o  | ← démarrage — vide les deux cartes de l'écran                          |
| LireManette       | 37 o  | ← bouton() — lit les huit boutons                                      |
| Tuiles            | 32 o  | ← CopierTuiles — 2 lettres sur 43                                      |
| "."               | 1 o   | ← texte() — un texte gravé                                             |

```

"."                1 o ← texte() – un texte gravé
Lettres gravées : A.
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche,
DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes
Fonctions du langage appelées :
  texte()          × 2    48 o sur place, dans main() + AdresseCase (10 o), EcrireTexte (10 o)
  effacer()        × 1    21 o sur place, dans main() + AdresseCase (10 o), EffacerCases (12 o)
  poser()          × 1    13 o sur place, dans main() + AdresseCase (10 o)
  bouton()         × 1     5 o sur place, dans main() + LireManette (37 o)
  image()          × 1     3 o sur place, dans main() + AttendreImage (28 o)
  images()         × 1     2 o sur place, dans chaque_minuteur()
Tes fonctions appelées :
  chaque_minuteur() × 2    21 o sur place, dans main()

```

## 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// ---- UNE LETTRE QUI TIRE : la place du A dans deux variables – plusieurs tirs ----
// Le 0.98 (ax et ay), avec trois tirs (le 0.97.1).

#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include <chaque>     // répond 1 toutes les n millisecondes
#include <bouton>     // lit un bouton de la manette
#include <texte>      // écrit un texte à l'écran
#include <effacer>    // efface des cases, ou tout le fond

```

## 2. UNE VARIABLE — AX

```
uint8_t ax = 2;           // le A : sa colonne...
```

## 3. UNE VARIABLE — AY

```
uint8_t ay = 8;           // ...et sa ligne
```

## 4. UN TABLEAU — TX

```
uint8_t tx[3];           // les colonnes des trois tirs
```

## 5. UN TABLEAU — VOLE

```
uint8_t vole[3];         // vole[i] = 1 : le tir n° i est en vol
```

## 6. UNE VARIABLE — LANCE

```
uint8_t lance = 0;
```

## 7. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    poser(ax, ay, ALPHABET[0]);

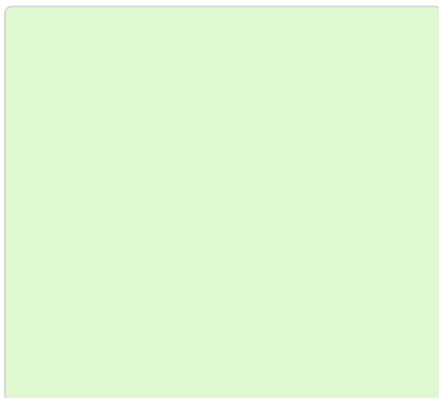
    while (true) {
        image();

        // ---- le bouton A lance un tir, au plus toutes les 250 ms (le 0.97.1) ----
        if (chaque(250)) {
            if (bouton(A)) {
                lance = 0;
                for (uint8_t i = 0; i < 3; i++) {
                    if (lance == 0 && vole[i] == 0) {
                        vole[i] = 1;
                        tx[i] = ax + 1;          // NOUVEAU : juste à droite du A, calculé (le 0.98)
                        texte(tx[i], ay, ".");   // sur la ligne du A
                        lance = 1;
                    }
                }
            }
        }

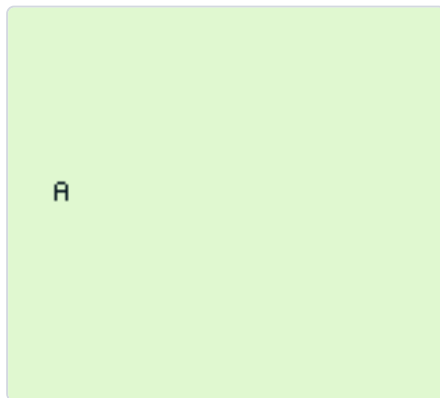
        // ---- chaque tir avance, sur la ligne du A ----
        if (chaque(100)) {
            for (uint8_t i = 0; i < 3; i++) {
                if (vole[i] == 1) {
                    effacer(tx[i], ay, 1);
                    tx[i] = tx[i] + 1;
                    if (tx[i] < 20) {
                        texte(tx[i], ay, ".");
                    } else {
                        vole[i] = 0;
                    }
                }
            }
        }
    }
}
```

## 3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1

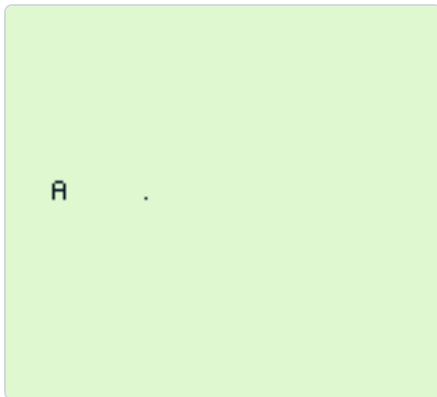


un tiers de seconde plus tard : le premier dessin est posé — image 20

## 4. Ce que les touches changent

---

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran a changé

## 5. Ce qu'on doit voir

---

Comme au 0.97.1 : le A en (2, 8), et le bouton A tenu lance une rafale de « . », jusqu'à trois en vol, qui partent juste à sa droite.

## 6. Ce que la console répond

---

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le A est en  $(ax, ay) = (2, 8)$
- ✓ le premier tir part juste à droite du A (colonne 4)
- ✓ A tenu : trois tirs en vol en même temps, pas plus (au plus 3)
- ✓ relâché : il ne reste que le A

---

Ensuite : **0.99. Une lettre qui tire — le A bouge, le tir garde sa ligne** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++